

第一关挑战 - AI 使用说明情况

一、使用的 AI 模型

本次挑战主要使用 ChatGPT (GPT-5.6 Sol) 作为辅助工具。AI 主要用于查询和解释环境配置步骤、辅助理解代码逻辑、分析终端运行结果，以及帮助我理解 SGLang、PagedAttention、RadixAttention 等比较陌生的概念。环境安装、命令执行、SGLang 服务启动、Mooncake trace 采样和 workload 运行均由我自己在本机完成。

二、我在哪些环节使用了 AI

第一个环节是环境部署。我把作业要求中的版本和模型信息提供给 AI，让它结合官方说明给出 WSL、Python、SGLang 0.5.14、Ray 2.56.0 和 Qwen/Qwen3-0.6B 的配置顺序。我按照步骤逐项执行，并通过 `/v1/models` 和 `/v1/chat/completions` 检查服务是否真正可用。

第二个环节是 workload 实验。我先把 Mooncake trace 中的样例数据提供给 AI，让它解释 timestamp、input_length、output_length 和 hash_ids 等字段。之后根据题目要求，参考 AI 给出的思路完成采样脚本和 workload 脚本：按照 input_length 构造 synthetic prompt，按照 output_length 控制生成长度，并用指数分布生成请求间隔来模拟 Poisson 到达过程。实验最后记录了 input tokens、output tokens、HTTP status 和 latency。

第三个环节是理论理解。对于 Prefill、Decode、KV Cache、PagedAttention、RadixAttention、prefix reuse 这些概念，我先通过 AI 的解释建立一个大致框架，再回到 SGLang、vLLM 和 Mooncake 的官方资料中核对。这样阅读论文时比较容易知道每个概念对应的是推理流程中的哪一部分。

三、使用过的代表性提示词

1. “怎么配环境，去官网阅读完使用说明后告诉我。”
2. “我已经能访问 `/v1/models`，并完成 `/v1/chat/completions` 请求，帮我判断这一步是否成功。”
3. “根据 input_length 和 output_length 构造 synthetic prompt，并用 Poisson 到达过程发送到 SGLang，记录 input/output tokens、status 和 latency。”
4. “解释一下为什么 Poisson 到达过程在代码里用指数分布生成请求间隔。”
5. “讲清楚 RadixAttention / RadixCache、page-sized KV cache、prefix reuse，以及它们和 vLLM PagedAttention 的关系和区别。”

四、AI 对我学习过程的帮助

AI 对我帮助比较大的一点，是把一个一开始看起来比较陌生的任务拆成了可以逐步验证的小步骤。比如环境部署时，不是一次把所有依赖都装完，而是先检查 GPU，再进入 WSL，确认 Python 环

境，最后启动 SGLang 服务并通过接口请求检查结果。这样每一步都有明确的成功标准，出现问题时也更容易判断是哪一层出了问题。

在理论部分，AI 更像是一个辅助解释工具。论文里关于 KV Cache 分页管理、共享前缀复用等内容比较抽象，经过解释以后，我能先知道它们分别是在解决显存管理问题还是重复计算问题，再去读官方资料时就不会只停留在记概念的层面。

五、AI 是否出现过误导，以及我是怎么处理的

这次使用过程中，我没有遇到会直接导致实验结果错误的明显误导，但我没有把 AI 的回答直接当成最终结论。我的做法主要有三种：第一，看给出的命令能不能在本机真正运行；第二，看接口返回的数据是否和预期一致；第三，涉及版本、参数和论文概念时，再去核对官方文档或论文。

例如 SGLang 官网当前文档对应的版本会持续更新，而本次作业明确要求使用 SGLang 0.5.14。如果直接照最新版文档操作，可能出现版本或参数不一致。因此我在询问时会明确限定作业指定版本，并通过 `sglang.__version__`、`ray.__version__` 以及实际推理结果再次确认。通过这次任务，我觉得 AI 更适合用来提高查资料、理解概念和调试代码的效率，但最后还是要靠实际运行结果和官方资料来验证。